

Starting with Selenium 4.6.0 and later, Selenium includes a feature called the **WebDriver Manager**, which automatically downloads and manages the appropriate WebDriver executables for your browser. This means you no longer need to manually download and set up WebDriver executables like `chromedriver.exe`, or `msedgedriver.exe`.

Why Does This Work?

When you create a `ChromeDriver` (or any other browser driver) instance:

```
ChromeDriver driver = new ChromeDriver();  
  
FirefoxDriver driver = new FirefoxDriver();  
  
EdgeDriver driver = new EdgeDriver();
```

Selenium's **WebDriver Manager** automatically:

1. Detects the version of the browser installed on your system.
2. Downloads the corresponding WebDriver binary (if not already downloaded).
3. Sets the system property for the driver (e.g., `webdriver.chrome.driver`) **behind the scenes**.

This eliminates the need to:

- Manually download WebDriver executables.
- Configure paths or system properties.

To start a **Selenium Java project in IntelliJ IDEA**, follow these steps:

◆ Step 1: Install IntelliJ IDEA and Java

1. Ensure **IntelliJ IDEA (Community or Ultimate)** is installed.
2. Install **Java (JDK 17 or later)** and set up **JAVA_HOME**.
3. Install **Apache Maven** (`mvn -version` should work in CMD).

◆ Step 2: Create a New Maven Project

1. **Open IntelliJ IDEA.**
2. Click **New Project** → Select **Maven**.
3. Set the project **name** (e.g., `SeleniumTestProject`).
4. Set the **Project SDK** (e.g., OpenJDK 23.0.2).
5. Click **Next** → **Finish**.

◆ Step 3: Update `pom.xml` to Include Selenium

1. Open `pom.xml`.
2. Add the **Selenium dependency** inside `<dependencies>`:

```
<dependencies>
  <!-- Selenium Java -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.16.1</version>
  </dependency>
</dependencies>
```

3. **Reload Maven:**
 - Open the **Maven tool window** in IntelliJ (on the right panel).
 - Click **Reload All Maven Projects**.

Java Project Execution

A Java program goes through multiple stages **from writing code to execution**. The process involves **compilation, bytecode generation, and execution by the JVM (Java Virtual Machine)**.

◆ Step 1: Writing Java Code

- ✓ A Java program starts as a **human-readable .java** file.
- ✓ Example: `HelloWorld.java`

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}
```

- ✓ You write the code in a text editor (**VS Code, IntelliJ, Notepad++**) or an IDE (**Eclipse, IntelliJ IDEA**).
-

◆ Step 2: Compiling Java Code (**javac**)

- ✓ The Java Compiler (**javac**) **converts the .java file into .class files** (bytecode).
- ✓ Run the command:

```
javac HelloWorld.java
```

- ✓ This generates a new file in the same directory:

```
HelloWorld.class
```

✓ Inside `HelloWorld.class` is Java Bytecode (not human-readable).

◆ Step 3: Bytecode Execution by the JVM (`java`)

- ✓ The JVM interprets the `.class` file and executes the program.
- ✓ Run the command:

```
java HelloWorld
```

- ✓ The JVM reads the bytecode and translates it into **machine code** for the OS.
- ✓ Output appears:

```
Hello, Java!
```

◆ Step 4: What Happens Inside the JVM?

When `java HelloWorld` runs, the **JVM** follows these steps:

1 Class Loader

- Loads `HelloWorld.class` into memory.
- Verifies the bytecode to prevent security risks.

2 Bytecode Interpretation (Execution Engine)

- Converts bytecode into machine instructions **specific to the OS**.
- Executes the program.

3 Just-In-Time (JIT) Compilation (Performance Boost)

- JVM **caches frequently used bytecode** as native machine code.
- This makes repeated executions **faster**.

What is a JAR File?

A **JAR (Java Archive)** file is a compressed package containing **Java classes, metadata, and resources**.

It is similar to a **ZIP file** but specifically for **Java programs**.

1. Components of a JAR File

A **.jar** file contains:

- **Compiled Java classes (.class files)**
- **Manifest file (META-INF/MANIFEST.MF)** for metadata
- **Resources (images, properties files, etc.)**

Example: The **Selenium JAR** contains:

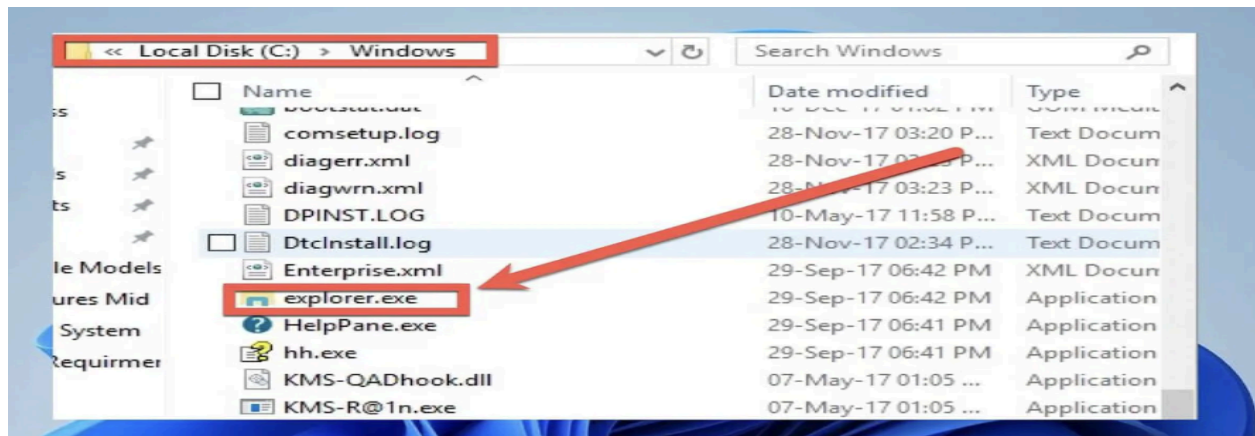
selenium-java-4.16.1.jar

```
├─ org/
|   └─ selenium/
|       ├── WebDriver.class
|       ├── ChromeDriver.class
|       └── FirefoxDriver.class
└─ META-INF/
    └─ MANIFEST.MF
```

This means **Selenium's compiled code is already inside** and ready to use.

A JAR file is like an "EXE" for Java—you can run it, use it as a library, or extract its contents, but **modifying it requires decompilation**. The extracted files will be **.class** files (compiled Java), not **.java** files.

An executable file (EXE file) is a computer file that contains an encoded sequence of instructions that the system can execute directly when the user clicks the file icon.

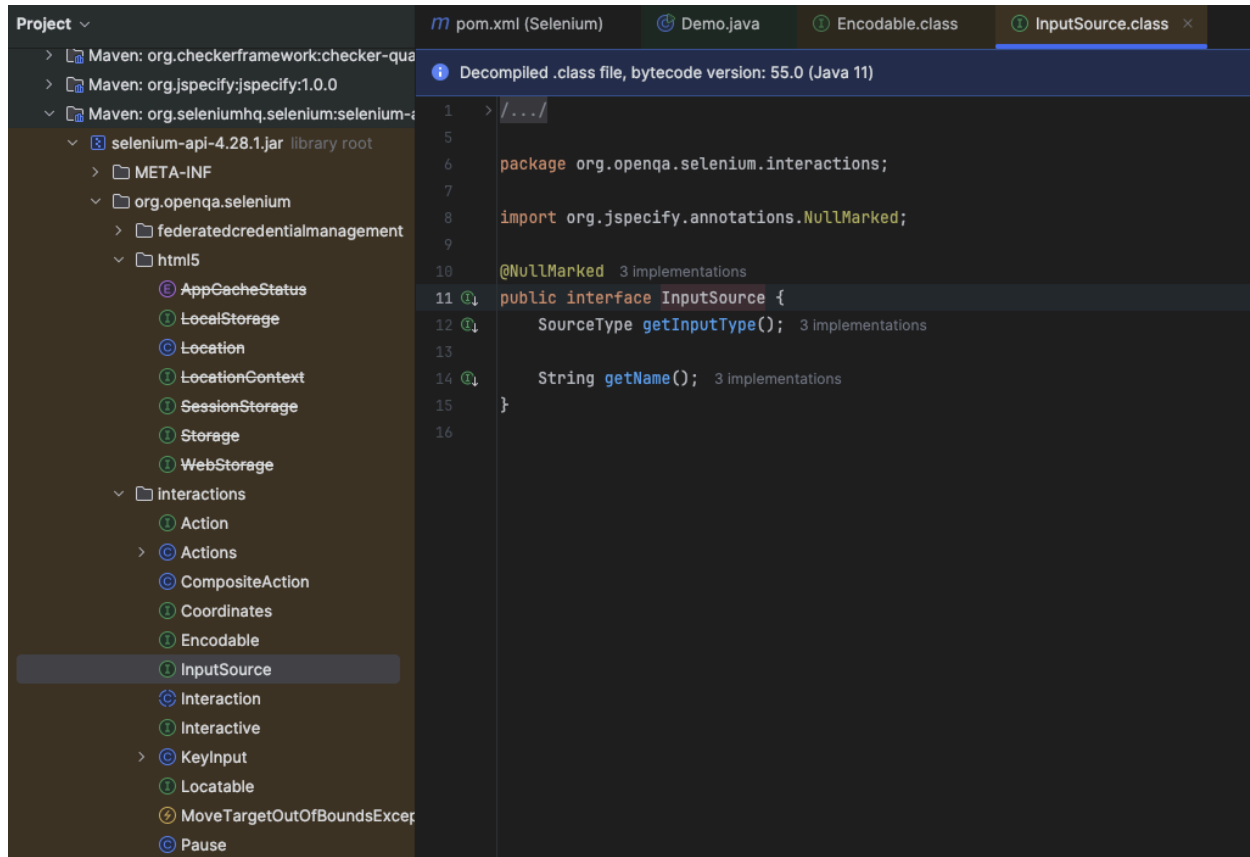


Feature	.java File (Source Code)	.class File (Compiled Bytecode)
Definition	Contains human-readable Java source code	Contains compiled Java bytecode (machine-readable)
Readability	Can be read and written by developers	Not human-readable (binary format)
Execution	Cannot be directly executed	Can be executed by the JVM (Java Virtual Machine)
Usage	Used to write and edit code	Used by the JVM to run programs

You won't see original **.java** files unless the JAR includes them. If you need to modify a project, it's **better to get the source code** rather than just the JAR.

A JAR is a compressed package of Java files (just like a ZIP file), but it is specifically structured for Java applications.

JAR files contain only compiled **.class** files, not **.java** files. However, you are still able to see Java source code from a JAR because of Java decompilers.



What is pom.xml in Maven?

pom.xml (Project Object Model) is the **configuration file** for a **Maven project**. It defines:

1. **Project details** (name, version, packaging).
2. **Dependencies** (like Selenium, TestNG).
3. **Build settings** (compiler version, plugins).
4. **Project structure** (sources, tests, resources).
5. **Execution lifecycle** (phases like clean, compile, test).

pom.xml is the heart of a Maven project.

It **defines dependencies, build configurations, and project metadata**.

It allows **automatic dependency management** (fetching Selenium, TestNG, etc.).

It makes **building and testing** Java projects easy (`mvn compile`, `mvn test`).

Without Maven, you'd manually download `.jar` files and manage updates.

◆ Key Sections in **pom.xml**

❶ **<groupId>, <artifactId>, <version>** (GAV Coordinates)

These define your project:

```
<groupId>com.example</groupId>    <!-- Your unique organization/project -->
<artifactId>SeleniumProject</artifactId> <!-- Project name -->
<version>1.0-SNAPSHOT</version>    <!-- Version of your application -->
```

❷ **<dependencies>** (Dependency Management)

This section tells Maven **which external libraries** (like Selenium, TestNG) to fetch:

```
<dependency>

    <groupId>org.seleniumhq.selenium</groupId>

    <artifactId>selenium-java</artifactId>

    <version>4.16.1</version>

</dependency>
```

When you add this, Maven automatically **downloads the required .jar files**.

❸ **<properties>** (Project Settings)

Defines **Java compiler settings**:

```
<properties>

    <maven.compiler.source>17</maven.compiler.source>

    <maven.compiler.target>17</maven.compiler.target>

</properties>
```

This ensures the project is compiled with **Java 17**.

4 **<build> (Build Plugins & Configuration)**

Configures **Maven plugins**, such as:

Compiler Plugin (ensures code compiles with a specific Java version):

```
<plugin>

    <groupId>org.apache.maven.plugins</groupId>

    <artifactId>maven-compiler-plugin</artifactId>

    <version>3.8.1</version>

    <configuration>

        <release>17</release>

    </configuration>

</plugin>
```

- **Surefire Plugin** (used to run tests automatically when using `mvn test`).

✓ Why Use Maven in Java Projects?

- ✓ **Automates dependency management** (no manual `.jar` downloads)
- ✓ **Builds and compiles** the project
- ✓ **Manages project structure** following best practices
- ✓ **Allows running Selenium tests** using TestNG/JUnit
- ✓ **Supports CI/CD integration**

In short, **Maven simplifies Selenium project setup, execution, and maintenance.**

1 Dependency Management

- Instead of manually downloading and adding `.jar` files for **Selenium**, **WebDriver**, and other libraries, Maven **automatically downloads** them from the **Maven Central Repository**.

The required dependencies are specified in the `pom.xml` file:

```
<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.16.1</version>  <!-- Maven downloads this version -->
  </dependency>
</dependencies>
```

- If a newer version of Selenium is released, you **just update the version in `pom.xml`**, and Maven will fetch the latest files.

2 Build Automation

- Maven **compiles** the project (`mvn compile`).
- It **packages** Java code into `.jar` or `.war` files.
- You can use `mvn clean install` to **build and install** the project.

3 Project Structure and Convention

Maven enforces a **standard project structure**:

SeleniumProject/

```
|— src/main/java      # Application source code
|— src/test/java      # Test classes
|— pom.xml            # Maven configuration file
|— target/            # Compiled output
└─ .m2/               # Local repository for dependencies
```

- This ensures **consistency** across different teams and projects.

4 Execution of Tests

Maven allows you to **run Selenium tests** using TestNG or JUnit:

```
mvn test
```

If you're using **TestNG**, you define it in `pom.xml`:

```
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.8.0</version>
  <scope>test</scope>
</dependency>
```

Then, you can execute Selenium test cases with:

```
mvn test
```

5 Integration with CI/CD

- **Maven integrates with Jenkins, GitHub Actions, and other CI/CD tools**, allowing automated execution of Selenium tests in a **continuous integration pipeline**.

What If We Manually Imported Selenium Instead of Using Maven?

1. Manual Import Process (Without Maven)

If Maven didn't exist, to use Selenium, we would need to:

1. **Manually download** the Selenium JAR from:
👉 <https://www.selenium.dev/downloads/>
2. **Extract and copy the JAR** to our project folder.
3. **Manually add it to the IntelliJ/Java classpath:**
 - In IntelliJ: **File** → **Project Structure** → **Libraries** → **Add Selenium JAR**
 - In Eclipse: **Right-click Project** → **Build Path** → **Add External JARs**
4. **Repeat the process** for all dependencies (e.g., WebDriverManager, TestNG, etc.).

2. Problems with Manual Import

✗ No Automatic Updates:

- If a new Selenium version is released, you must **manually download and replace** the JAR.

✗ Dependency Management is Hard:

- Selenium depends on **multiple other JARs** (e.g., WebDriverManager, Apache Commons).
- **Finding and downloading each dependency manually is time-consuming.**

✗ Inconsistent Builds:

- If a teammate downloads **a different version**, the code may break.

✗ No Easy CI/CD Integration:

- CI/CD pipelines (e.g., Jenkins, GitHub Actions) rely on **automated dependency fetching**.
- Without Maven, we'd need to **upload JARs manually to servers**.

How Does Maven Fetch Selenium Libraries Automatically?

When you specify:

```
<dependency>

    <groupId>org.seleniumhq.selenium</groupId>

    <artifactId>selenium-java</artifactId>

    <version>4.16.1</version>

</dependency>
```

Maven **performs these steps**:

1. **Check Local Repository (.m2 folder)**
 - If `selenium-java-4.16.1.jar` exists in `~/.m2/repository/`, Maven **uses** the cached version.
2. **Download from Maven Central (if not found locally)**

If the JAR is **not in .m2**, Maven **fetches it** from the **Maven Central Repository**:

<https://repo.maven.apache.org/maven2/org/seleniumhq/selenium/selenium-java/4.16.1/>

- The downloaded `.jar` is stored in `.m2`.
3. **Link the JAR to Your Project**
 - IntelliJ IDEA and Maven will **automatically add the Selenium JAR to the classpath**.

👉 This eliminates the need to manually download and import `.jar` files!

What is the Maven Repository?

The **Maven Repository** is a centralized storage system for Java libraries and dependencies. It contains thousands of precompiled **JAR files** (Java libraries) that developers can use in their projects.

There are **three types** of Maven repositories:

1. Maven Central Repository (Default)

- A public repository where **official** Java libraries (e.g., Selenium, TestNG, JUnit) are stored.
- When you specify a dependency in `pom.xml`, Maven automatically **downloads it** from [Maven Central](#).

2. Local Repository (~/.m2/repository/)

Once a dependency is downloaded, Maven **caches** it in your system inside:

bash

CopyEdit

`C:\Users\YourUser\.m2\repository` (Windows)

`~/.m2/repository` (Linux/Mac)

- This prevents **re-downloading** the same library every time.

3. Remote Repository

- Some organizations or companies have their **private repositories** to store internal libraries.
- Example: **Nexus Repository Manager, Artifactory**.